

Ain Shams University
Faculty of Engineering
Mechatronics Engineering Department

Collaborative Robotic Arms Digital-Twin Enabled

Graduation Project Thesis

Submitted in partial fulfillment of the requirements for the
B.Sc. degree in Mechatronics Engineering

Team Members:

Marwan Mahmoud	Mohamed Tarek
Mariam El Sebaey	Mohamed Alaa
Omar Magdy	Ingy Ahmed
Yehia Raouf	Kareem Saleh

Supervisors:

Dr. Mohamed Omar
Dr. Shady Ahmed
Eng. Dina Zakaria

Academic Year: 2025–2026
Cairo, Egypt

Declaration

We/I hereby certify that this project submitted as part of our/my partial fulfillment of BSc in Mechatronics Engineering is entirely our/my own work, that we/I have exercised reasonable care to ensure its originality, and does not to the best of our/my knowledge breach any copyrighted materials, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our/my work.

Signed by:

Name	Signature
Ingy Ahmed Sherif	
Kareem Saleh Hassan	
Mariam Mohamed El Sebaey	
Marwan Mahmoud Ali	
Mohamed Alaa Abdelkareem	
Mohamed Tarek Abdelwahab	
Omar Magdy Abdelsattar	
Yahia Mohamed Abdelraouf	

Date: January 29, 2026

Acknowledgment

The team would like to thank our supervisors,

Abstract

This thesis presents a Digital-Twin-enabled collaborative robotic assembly system integrating an ABB industrial manipulator and an AR4 robotic arm to execute efficient brick assembly through coordinated perception, grasping, handover, and placement. The proposed system combines global scene understanding from an Intel RealSense camera with local “eye-in-hand” sensing using an ESP32-CAM mounted on the AR4, enabling both coarse positioning and fine alignment during grasping via Image-Based Visual Servoing (IBVS). A structured state machine governs end-to-end task allocation and execution, selecting between direct assembly and cross-side handover based on brick and robot allocation logic derived from GUI/database input and global perception.

To achieve accurate interaction in a shared workspace, calibration is treated as a core requirement. AR4 calibration is performed by collecting data at multiple X–Y locations at two height levels (0 mm and 50 mm) and fitting mapping functions using multiple approaches (Radial Basis Function (RBF), linear, and polynomial). Polynomial regression demonstrated superior performance and was validated using training/testing splits. Similarly, global camera calibration based on polynomial regression is applied using samples across varying X, Y, and Z coordinates to enable reliable estimation of brick world pose across the entire table. Perception is implemented through YOLOv8 segmentation for brick and feature detection, while grasp selection is addressed using a transformer-based encoder–decoder trained with both successful grasps (True Grasps) and failure/avoidance cases (False Grasps), enabling negative reinforcement to reduce unstable grasp predictions. During fine control, homography is used to map feature points between the RealSense global view and the ESP32-CAM local view, simplifying tracking and improving approach robustness.

Experimental results demonstrate improved positional accuracy using polynomial calibration, effective brick detection performance using segmentation, increased grasp stability using the True/False labeling strategy, and robust end-to-end execution under a state-machine framework. The thesis concludes with limitations and future directions, including improved domain adaptation, force-aware handovers, and enhanced Digital Twin synchronization for higher fidelity planning and monitoring.

Contents

1	Introduction	8
1.1	Background and Motivation	8
1.2	Problem Statement	8
1.3	Project Objectives	9
1.4	Project Scope	9
1.5	Key Contributions	10
1.6	Thesis Organization	10
2	System Design and Methodology	11
2.1	System Overview	11
2.1.1	System Architecture Overview	11
2.1.2	Layer 1: High-Level User Interface (GUI) Layer	11
2.1.3	Layer 2: Supervisor Layer	13
2.1.4	Layer 3: Mid-Level Execution Layer (ROS Modules)	16
2.1.5	Layer 4: Low-Level Hardware Abstraction Layer	20
2.1.6	Layer 5: Physical Hardware Layer	22
2.1.7	Cross-Layer Communication and Data Flow	23
2.1.8	Synchronization and Timing	24
2.1.9	Simulation Environment	24
2.1.10	System Properties and Design Considerations	24
2.2	AR4 Restoration and Hardware Modifications	25
2.2.1	Joint 3 Mechanical Transmission Upgrade	25
2.2.2	Shaft Key Re-Manufacturing in Joint 3	26



2.2.3	Encoder Signal Noise in 5th and 6th joint	26
2.2.4	Joint 1 Backlash Due to Missing Fasteners	26
2.2.5	Summary	27
2.3	Cartesian Calibration of the AR4 Collaborative Robot	28
2.3.1	Experimental Setup	28
2.3.2	Cartesian Error Characterization	28
2.3.3	Calibration Method Selection	28
2.3.4	Polynomial Calibration Model	29
2.3.5	Calibration Results	29
2.3.6	Error Surface Analysis	30
2.3.7	Quantitative Evaluation	32
2.4	Assembly Supervisor and State Machine Control	33
2.4.1	Overview	33
2.4.2	Supervisor Architecture and Responsibilities	33
2.4.3	ROS 2 Communication: Services and Actions	34
2.4.4	Transform (TF2) Management and Coordinate Frames	35
2.4.5	Detailed State Machine Analysis	37
2.4.6	Error Handling and Recovery	45
2.4.7	Concurrency and Threading Model	46
2.4.8	State Machine Timing and Scheduling	46
2.4.9	Hardware and Sensor Integration	47
2.4.10	Data Flow and Message Passing	48
2.4.11	Debugging and Logging	49
2.4.12	Summary	49
2.5	Introduction	50
2.6	Literature Review	50
2.6.1	Visual Servo Control Foundations	50



2.6.2	The Pinhole Camera Model	50
2.6.3	The Pinhole Camera Model	51
2.6.4	Position-Based Visual Servo (PBVS)	51
2.6.5	Image-Based Visual Servo (IBVS)	52
2.6.6	Comparative Analysis: IBVS vs. PBVS	53
2.6.7	Homography-Based Euclidean Reconstruction	53
2.7	General Concept and Control Flow	54
2.8	Implementation Details	55
2.8.1	Local Perception and Depth Estimation	55
2.8.2	Global Perception and Homography Matrix Calculation	56
2.8.3	Transformation Logic	56
2.8.4	Control Logic and Feature Generation	56
2.9	Results and Evaluation	57

Chapter 1

Introduction

1.1 Background and Motivation

Collaborative robotic systems are increasingly adopted in modern automation to improve throughput, flexibility, and safety in assembly workflows. While a single industrial robot can achieve high repeatability, multi-robot collaboration enables task decomposition, parallelism, and improved workspace coverage.

Despite these advantages, multi-robot collaboration introduces substantial technical challenges. These include:

- Accurate calibration between sensing systems and robotic manipulators.
- Reliable perception under dynamic and uncertain real-world conditions.
- Robust grasp planning to avoid object slippage or collision.
- Stable closed-loop control for fine alignment during assembly.
- Coordination of system-level logic to manage roles, task handovers, and execution uncertainties.

This project addresses these challenges by implementing a two-robot collaborative assembly system, integrating an ABB industrial robot and an AR4 research robot to perform brick assembly tasks. The system leverages Digital Twin technology to provide visualization, monitoring, and structured integration of perception, planning, and execution modules.

1.2 Problem Statement

The primary problem addressed in this project is the design and implementation of a robust, two-robot collaborative assembly system capable of:

1. Detecting and localizing bricks within a shared workspace using global perception.
2. Selecting stable grasps and avoiding unstable configurations via learned models.
3. Executing safe and accurate grasping using hybrid control strategies (point control combined with image-based visual servoing).

4. Performing reliable handovers from the AR4 to the ABB robot when bricks need to cross workspace sides.
5. Maintaining a consistent world coordinate system across the workspace through calibration of cameras and robots.
6. Coordinating the full workflow via a state machine with well-defined transitions and recovery logic.

1.3 Project Objectives

The objectives of this thesis are summarized as follows:

- Develop a state machine to govern setup, task allocation, grasping, handover, and assembly operations reliably.
- Implement robust object detection using YOLOv8 segmentation for bricks and workspace features.
- Implement grasp detection using a transformer-based encoder-decoder trained with stable and unstable grasps to improve reliability.
- Implement fine alignment for AR4 grasping using image-based visual servoing aided by homography mapping between global and local camera views.
- Achieve accurate AR4 position correction using data-driven calibration techniques (tested: radial basis function, linear, polynomial; best performance: polynomial regression).
- Calibrate the global camera-to-world mapping for reliable brick pose estimation across the entire workspace.
- Develop a Digital Twin-enabled architecture integrating perception, decision logic, and robot execution.

1.4 Project Scope

This thesis focuses on the perception-to-action integration for collaborative assembly using two robotic arms and two camera systems (global and eye-in-hand). The scope includes:

- System calibration and camera-to-robot alignment.
- Object detection and grasp pose estimation.
- Visual servoing and hybrid control for precise manipulation.
- Handover logic between robots.
- Digital Twin integration for monitoring and validation.

The thesis does **not** include:

- Detailed mechanical design of the robots.
- Proprietary low-level controller programming of the ABB robot.
- Safety certification processes beyond safe experimental operation.

1.5 Key Contributions

The main contributions of this project are:

1. Implementation of a Digital Twin-enabled multi-robot assembly system for visualization, validation, and monitoring.
2. Development of a polynomial-based calibration method for precise AR4 positioning and global workspace mapping.
3. Integration of YOLOv8-based object detection for brick and feature localization.
4. Implementation of a transformer-based grasp detection network for improved grasp stability.
5. Hybrid control strategy combining point control and image-based visual servoing for fine alignment.
6. Reliable handover and task coordination using a state machine with recovery mechanisms.

1.6 Thesis Organization

This thesis is organized as follows:

- **Chapter 2** reviews literature on collaborative robotics, multi-robot handover, visual servoing, grasp detection, and Digital Twin integration.
- **Chapter 3** describes the system architecture, including calibration, perception, grasping, control, and workflow management.
- **Chapter 4** presents the experimental implementation, evaluation of calibration accuracy, perception performance, grasp prediction quality, and workflow reliability.
- **Chapter 5** concludes the thesis, summarizes key findings, and discusses potential future enhancements.

Chapter 2

System Design and Methodology

2.1 System Overview

This section presents a comprehensive overview of the collaborative dual-arm robotic brick assembly system. The architecture is designed hierarchically with five distinct layers, each with specific responsibilities and interactions. Figure ?? illustrates the complete system architecture, demonstrating the top-down control flow from user input through hardware execution.

2.1.1 System Architecture Overview

The system follows a layered architecture pattern, which enables modularity, maintainability, and clear separation of concerns. The architecture consists of five primary layers:

1. **High-Level User Interface (GUI) Layer:** User interaction and task specification
2. **Supervisor Layer:** Task allocation, workflow management, and synchronization
3. **Mid-Level Execution (ROS) Layer:** Control, planning, and perception modules
4. **Low-Level Hardware Abstraction Layer:** Driver abstraction and hardware communication
5. **Physical Hardware Layer:** Robotic arms, grippers, cameras, and embedded systems

Each layer abstracts the complexity of lower layers and provides a well-defined interface to higher layers, facilitating independent development and testing of components.

2.1.2 Layer 1: High-Level User Interface (GUI) Layer

Overview and Purpose

The GUI layer serves as the primary interface for specifying desired brick assembly configurations, it is used exclusively to define the target assembly pattern that the system should achieve. The GUI is the entry point for task specification and enables operators to intuitively construct complex brick arrangements without low-level programming.

Key Components

Grid-Based Assembly Editor: The GUI provides a grid-based interface where operators place bricks in desired positions (rows and columns) to define the target assembly. The interface supports:

- **Four Brick Types:** Selection from a library of distinct brick geometries
- **Rotational Control:** Each brick can be rotated in four main angles (0° , 90° , 180° , 270°) to define orientation
- **Spatial Placement:** Drag-and-drop placement on a 2D grid representing the assembly workspace
- **Grid Coordinates:** Each position maps to specific row and column indices defining the target location

Task Specification Output: Once the user defines the desired arrangement, the GUI generates a structured task specification including:

- Brick type for each grid position
- Rotation angle for each brick (0° , 90° , 180° , 270°)
- Row and column indices defining placement locations on the grid

This task specification is submitted to the Firestore database as a JSON document for persistent storage.

Implementation Details

The GUI is implemented as a web application that communicates with the robotic system through a cloud database backend. The architecture includes:

- **Web Application Frontend:** A responsive web interface (JavaScript) that runs in a user's browser, providing the grid-based brick placement editor
- **Firestore Database Backend:** The GUI submits task specifications to a Google Firestore database in the following structure:
 - JSON document containing the complete task specification
 - Fields for each grid position: brick type and rotation angle
 - Row and column indices
 - Timestamp and task metadata for tracking
- **ROS 2 Database Bridge with Assembly Allocator:** A dedicated ROS 2 Python node that serves as the critical interface between the GUI specification and robot execution. The node operates in two stages:

1. Task Retrieval and Parsing:

- Continuously monitors the Firestore database for new task submissions
- Fetches the JSON document using configured Firestore keys
- Extracts row/column indices, brick types, and rotations from the JSON
- Performs coordinate transformation from grid indices to desired 3D workspace coordinates using known grid spacing, brick dimensions, and workspace origin offset

2. Assembly Allocator:

- Subscribes to brick detections from the Perception Module (current brick positions in 3D workspace)
- Compares the desired arrangement (from GUI) against the available detected bricks in the current scene
- Validates that all required bricks are present and properly detected with sufficient confidence
- Performs spatial matching: associates each desired grid position with a detected brick instance
- Upon successful validation, generates an assembly plan containing:
 - * For each brick: current 3D position (XYZ) acquired from detection
 - * For each brick: desired 3D position (XYZ) computed from GUI specification
 - * Brick type and required rotation

The assembly plan is published to the Supervisor, which then coordinates robot execution to move each brick from its current detected position to the desired position.

- **Decoupling Advantage:** By using Firestore as an intermediary, the GUI is completely decoupled from the ROS 2 system. The web application requires no ROS knowledge or configuration, and can be deployed independently on any web server. The database acts as a reliable, persistent message queue that survives temporary system disconnections.
- **No Real-Time Feedback:** The GUI remains purely an input specification tool; it receives no real-time feedback from the robot system

The Supervisor and Perception modules are responsible for determining the current state of the workspace and planning actions to achieve the GUI-specified configuration retrieved from the Firestore database.

2.1.3 Layer 2: Supervisor Layer

Overview and Purpose

The Supervisor layer represents the core orchestration component of the system. It manages the coordination between two independent robotic arms (ABB and AR4), handles workflow synchronization, monitors feedback from lower layers, and implements recovery mechanisms when anomalies occur. The Supervisor is responsible for translating high-level task specifications into concrete control commands for individual robots.

Key Responsibilities

Assembly Plan Retrieval and Task Dequeueing: The Supervisor retrieves the assembly plan from the ROS 2 Database Bridge node (via service call). This plan contains the ordered sequence of bricks to assemble, with both current and desired positions. The Supervisor maintains an assembly queue and sequentially dequeues bricks for processing.

Perception Integration and Transform Management: The Supervisor coordinates global brick detection from the RealSense camera:

- Invokes the detection service to obtain brick poses in the camera's optical frame
- Applies coordinate frame transformations (TF2) to convert all detected brick poses to the ABB base frame
- Computes and transforms the handover pose (neutral zone between the two robots)
- Validates spatial consistency of all detected poses

Grasp Planning and Selection: For each brick to be grasped:

- Queries the grasp inference service to obtain the optimal grasp candidate for the current brick
- Receives grasp pose (with quality metrics) in the camera frame
- Transforms grasp pose to the ABB base frame and applies safety constraints (e.g., minimum height clamps)

Motion Execution and Robot Coordination: The Supervisor issues concrete motion commands to both robots via action servers:

- **AR4 Execution:** Submits point control actions for approach, grasp, and placement motions; optionally triggers visual servoing (IBVS) via the ESP32 wrist camera for precision control
- **ABB Execution:** Submits high-level task actions (pick, place, pick-from-handover) that are internally managed by the ABB controller
- **Handover Coordination:** If handover strategy is selected, orchestrates synchronized execution where AR4 picks a brick and ABB receives it at the handover pose

Strategy Selection and Allocation Logic: The Supervisor decides between two execution strategies:

- **Direct Assembly:** Robot directly picks and places a brick when reachable and collision-free
- **Handover Strategy:** AR4 picks the brick and passes it to the ABB at the handover zone for placement, enabling access to areas unreachable by AR4 alone

State Machine Management: The Supervisor implements a multi-state finite state machine that coordinates the complete workflow:

- **INIT:** Retrieve assembly plan from Database Bridge
- **DETECT:** Perform global brick detection and transform poses
- **PROCESS_NEXT:** Dequeue next brick from assembly queue
- **GRASP_PIPELINE:** Query grasp inference service for selected brick
- **Execution States:** EXECUTE_AR4_DIRECT, AR4_PICK_FOR_HANOVER, EXECUTE_ABB_PICK, EXECUTE_ABB_PLACE, HANOVER_EXECUTION
- **DONE:** Assembly complete; system idle

Error Handling and Recovery: When anomalies occur:

- Detects service failures and timeouts
- Implements retry logic with controlled backoff
- Logs detailed error information for debugging

Implementation Details

The Supervisor is implemented as a ROS2 Python node that maintains continuous communication with all lower-layer modules. It uses ROS2 services for synchronous task initiation and topics for asynchronous feedback. Key implementation aspects include:

- **Namespaced Robot Control:** Uses ROS 2 namespace hierarchies (`/abb/`, `/ar4/`) to maintain separate action clients and service clients for each robot, enabling independent command submission
- **Sequential Execution Model:** The current implementation operates sequentially—while one arm (ABB or AR4) executes a motion, the other arm remains idle.
- **Action-Based Synchronization:** Uses ROS 2 action servers for non-blocking, feedback-capable task execution. The Supervisor submits action goals and blocks until completion, ensuring tasks execute serially
- **Timing and Synchronization:** Implements timeout mechanisms and heartbeat monitoring to detect stalled or crashed components.
- **Multi-Threaded Executor:** Employs a reentrant callback group allowing concurrent handling of service requests and action goal submissions without blocking the main state machine loop

2.1.4 Layer 3: Mid-Level Execution Layer (ROS Modules)

The mid-level execution layer consists of three independent but interconnected ROS 2 modules: Control, Planning, and Perception. These modules implement the core algorithmic components of the system and directly interface with hardware through the abstraction layer below.

3.1 Control Module

Overview: The Control Module translates high-level motion objectives into low-level joint commands. It implements multiple control strategies tailored to different task phases and ensures robust execution despite environmental variations.

Key Components:

- **Point Controllers:** Implemented as ROS 2 action servers that receive desired end-effector coordinates (Cartesian pose) from the Supervisor. The Point Controllers:
 - Accept a goal containing target coordinates (X, Y, Z, orientation)
 - Delegate motion planning and trajectory computation to MoveIt2
 - Execute the planned trajectory on the physical robot via the Hardware Abstraction Layer
 - Provide feedback on execution progress and result status
 - Do not directly compute IK or trajectories; MoveIt2 handles all planning logic
- **IBVS (Image-Based Visual Servoing) Controller:** For the AR4 arm, the IBVS controller provides visual feedback correction during pick-and-place operations. This is critical for:
 - Compensating for small calibration errors
 - Correcting for environmental variations (brick surface reflectivity, lighting)
 - Fine-tuning gripper approach trajectories
 - Providing closed-loop feedback during manipulation

The IBVS uses the ESP32 wrist-mounted camera as feedback and implements visual feature tracking (edges, corners) to maintain precise alignment with the target brick. The visual error is converted to velocity commands using the camera Jacobian.

- **Gripper Controllers:** Manage pneumatic (ABB) and servo-based (AR4) grippers with:
 - Open/close state management
 - Brick hold verification

Implementation Details: The Control Module runs as separate ROS 2 action servers for each robot (one per arm). Each action server:

- Receives motion goals (target coordinates) from the Supervisor
- Calls MoveIt2's planning and execution interfaces to compute and execute trajectories

- Monitors execution status and provides feedback to the Supervisor
- Publishes low-level joint commands at rates of 100-500 Hz depending on the control strategy
- Implements safety monitoring for joint limits, velocity limits, and collision detection

3.2 Planning Module

Overview: The Planning Module computes collision-free trajectories and solves inverse kinematic problems. It serves as the interface between high-level task objectives and concrete motion commands, ensuring that all computed motions are feasible and safe.

Key Components:

- **Inverse Kinematics (IK) Solvers:** Two solvers are employed with selective switching based on active control mode:
 - **IKFast (Analytical Solver):** The default solver for both AR4 and ABB arms. Provides fast, exact solutions by leveraging pre-computed closed-form solutions offline, enabling real-time IK queries with multiple solution branches
 - **KDL (Numerical Solver):** Used exclusively for AR4 when Image-Based Visual Servoing (IBVS) is active. The iterative numerical approach allows continuous refinement of joint configurations in response to visual feedback, which is essential for the precision correction that IBVS provides
 - **Solver Switching Logic:** When IBVS is activated on AR4, the IK solver switches from IKFast to KDL to support real-time visual servo corrections. When IBVS is inactive, both robots use IKFast for rapid planning and execution

IK solvers are critical for:

- Converting Cartesian end-effector poses (desired pick/place positions) to joint configurations
- Finding multiple solution branches to select collision-free configurations
- Incorporating constraints (joint limits, reachability limits)
- **Path Planning (RRTconnect):** Uses MoveIt2's interface to OMPL (Open Motion Planning Library) with the RRTconnect algorithm as the primary motion planner. RRTconnect:
 - Employs bi-directional Rapidly-Exploring Random Trees (RRT) from both start and goal configurations
 - Grows two trees simultaneously, one from the start state and one from the goal state, significantly accelerating convergence
 - Effective for high-dimensional configuration spaces, suitable for 6-DOF robotic arms
 - Naturally handles complex environments with obstacles and narrow passages through randomized exploration
 - Produces kinematically feasible paths without requiring analytical solutions

The path planner operates on a collision model that includes:

- Robot self-collision models (defined in URDF)
- Environment obstacles (static table, fixed structures)
- Other robot (for collision avoidance between arms)
- **Collision Avoidance:** Beyond path planning, collision detection occurs at two levels:
 - **Planning Phase:** OMPL checks collisions during trajectory sampling
 - **Execution Phase:** Real-time collision detection using joint feedback validates that actual motion matches planned motion
- **Trajectory Generation:** Converts paths (sequences of configurations) into time-parameterized trajectories respecting velocity and acceleration limits. MoveIt2 handles trajectory parameterization using:
 - Time-optimal trajectory parameterization for efficient motion
 - Joint velocity and acceleration limits configured per robot
 - Gripper-specific constraints

Implementation Details: The Planning Module is implemented as a ROS2 C++ node with direct MoveIt2 integration. MoveIt2 encapsulates RRTconnect planning, IK solving, and trajectory parameterization. Key architectural aspects:

- **Asynchronous Planning:** Planning requests are handled asynchronously to avoid blocking the Supervisor
- **Planning Scene Updates:** Maintains a collision world model that is continuously updated with:
 - Detected object positions from the Perception Module
 - Current robot states (to detect collisions in progress)
 - Manual environment updates from the GUI
- **Solver Selection Logic:** AR4 dynamically switches between IKFast (default) and KDL (when IBVS is active) based on control mode:
 - **Standard Execution:** Both AR4 and ABB use IKFast for rapid, deterministic inverse kinematics
 - **Visual Servo Active:** AR4 switches to KDL to enable continuous refinement guided by visual feedback from the ESP32 wrist camera. KDL's iterative nature allows online adjustment of joint configurations in response to perceived visual errors, whereas IKFast's pre-computed solutions are optimized for bulk solution generation rather than online refinement

3.3 Perception Module

Overview: The Perception Module processes visual data from the RealSense camera to understand the environment, detect objects, estimate grasp points, and provide visual feedback. It serves as the sensory foundation for autonomous decision-making throughout the system.

Key Components:

- **Object Detection (YOLOv8 Segmentation):** Uses the state-of-the-art YOLOv8 segmentation model to:
 - Detect brick instances in the scene
 - Provide pixel-level segmentation masks
 - Output: Bounding boxes, class labels, confidence scores, and segmentation masks
 - **Metrics:** Performance is evaluated using mean Average Precision (mAP) and Intersection over Union (IoU)
 - * mAP: Evaluates detection quality across all confidence thresholds
 - * IoU: Measures segmentation mask accuracy

The YOLOv8 model is deployed with the Realsense scene camera (top-down view) to maintain a global view of brick locations and availability.

- **Grasp Point Generation (Transformer-Based Learning):** Estimates optimal gripper approach poses for detected bricks. The system employs:
 - **Vision Transformer (ViT) Architecture:** Encodes multi-scale image features to understand local geometric properties
 - **Grasp Quality Estimation:** Learns to predict grasp success probability
 - **6-DOF Pose Prediction:** Estimates gripper orientation and position relative to the brick

Grasp learning is trained on:

- Synthetic grasping datasets (procedurally generated)
- Real-world grasp attempts (labeled for success/failure)
- Domain adaptation techniques to transfer synthetic knowledge to real scenarios

The output is a ranked list of candidate grasps with confidence scores, allowing the Planning Module to select collision-free grasps.

- **Grasp Pose Estimation:** Converts predicted gripper poses (relative to camera frame) into world coordinates:
 - Camera calibration: Intrinsic parameters determined offline
 - Camera-to-base transformation: Computed from known camera mounting geometry
 - Coordinate transformation pipeline: RGB image → camera frame → robot base frame
- **Planar Homography (Optional Disambiguation):** For challenging scenarios (occluded bricks, cluttered scenes), planar homography estimates 3D brick locations from 2D image coordinates assuming bricks lie on a known plane (the work surface):

- Homography matrix H is computed from known point correspondences
- 2D image points are lifted to 3D using the plane constraint

Implementation Details: The Perception Module is implemented as a ROS 2 Python node with GPU-accelerated inference:

- **Camera Feed Source:** Subscribes exclusively to:
 - Realsense scene camera feed (for full YOLOv8 + grasp inference processing)
- **Inference Pipeline:**
 1. Receive RGB frame from Realsense camera
 2. Preprocess: Resize, normalize to YOLOv8 input format
 3. Run YOLOv8 segmentation inference (GPU)
 4. For each detected brick: Extract ROI, run grasp transformer
 5. Convert output to world frame using calibrated transformations
 6. Publish detected objects and grasp candidates
- **Latency Optimization:**
 - Batched inference when multiple detections occur
 - Asynchronous inference using thread pools to avoid blocking the main ROS loop
 - Frame skipping for non-critical grasp updates
- **Fallback Strategies:**
 - If grasp learning fails: Use geometric grasp estimators (surface normal analysis)
 - If YOLOv8 fails (rare cases): Switch to simpler detection methods or manual intervention

2.1.5 Layer 4: Low-Level Hardware Abstraction Layer

Overview and Purpose

The Hardware Abstraction Layer (HAL) decouples high-level ROS modules from hardware-specific communication details. It provides standardized interfaces for hardware control and feedback, enabling seamless swapping of hardware implementations without changes to higher layers.

Key Components

Encoder Processing: Reads joint angle feedback from:

- Robot joint encoders (via proprietary robot communication protocols)

Motor Drivers: Interface with:

- AR4 servo motors (for joints and gripper)
- Motor control boards managing PWM signals and current feedback

Camera Drivers: ROS camera interface nodes for:

- Realsense SDK integration (color, depth, point clouds)
- ESP32 camera module

Communication Protocols: Manages hardware communication over:

- UART/Serial (AR4 Communication)
- Ethernet (ABB Communication)
- USB (Camera Interfaces)

Implementation Details

The HAL is implemented as a set of specialized ROS2 nodes:

- **Robot Driver Nodes:** Each robot (ABB, AR4) has a dedicated driver node that:
 - Communicates with the robot controller using vendor-specific APIs
 - Translates ROS command messages to robot commands
 - Publishes robot state (joint positions, velocities, forces) on standardized ROS topics
 - Implements hardware safety checks (joint limits, fault detection)
- **Camera Driver Nodes:**
 - Realsense driver: Publishes RGB/Depth at configurable rates
 - ESP32 driver: Custom firmware and ROS bridge for low-latency video streaming
- **Standardized Topic Interfaces:** All hardware communication is abstracted to standard ROS2 topics:
 - `/robot_name/joint_states`: Joint feedback
 - `/robot_name/command/joint_trajectory`: Trajectory commands
 - `/gripper/state`: Gripper status
 - `/camera/rgb/image_raw`: Camera feeds

2.1.6 Layer 5: Physical Hardware Layer

Overview

The Physical Hardware Layer comprises the actual robotic systems, sensors, and actuators that interact with the physical world.

Key Hardware Components

ABB Robotic Arm: Industrial 6-DOF collaborative arm with:

- High payload capacity
- Precise joint control via proprietary controller
- Pneumatic end-effector gripper for power grasping
- Integrated force/torque feedback (optional)

AR4 Robotic Arm: Custom 6-DOF robotic arm with:

- Servo-based joints (high precision, lower payload)
- Modular servo-gripper with integrated sensors
- ESP32 wrist-mounted camera for close-range visual servoing
- Lighter weight, suitable for fine manipulation tasks

Realsense Scene Camera: Intel Realsense D435i camera:

- Mounted at top of workspace, looking down
- Provides color image for YOLOv8 detection
- Provides depth for 3D reconstruction

ESP32 Wrist Camera: Custom lightweight camera module mounted on AR4 wrist:

- OV2640 Wide angle lense 2MP camera sensor
- Real-time video streaming via WiFi
- Low power consumption (suitable for wrist mounting)
- Used exclusively for IBVS feedback

Gripper Hardware:

- ABB gripper: Pneumatic dual-finger, open/close control
- AR4 gripper: Servo-driven with force feedback and state feedback

2.1.7 Cross-Layer Communication and Data Flow

To complete the architectural understanding, this section describes how data and commands flow through the layered system:

Top-Down Command Flow

1. **User Input:** Operator specifies a brick picking and placing task in the GUI
2. **Supervisor Interpretation:** Supervisor decomposes task into concrete motion commands
3. **Planning:** Planning Module computes collision-free trajectories using IK and OMPL solvers
4. **Control:** Control Module executes trajectories with real-time feedback from encoders
5. **Hardware Execution:** Motors and grippers execute commands, generating motion

Bottom-Up Feedback Flow

1. **Sensor Readings:** Hardware layer generates continuous sensor data (joint angles, camera frames)
2. **Hardware Abstraction:** HAL layer processes raw sensor data and publishes standardized ROS messages
3. **Control Feedback:** Control Module uses feedback for closed-loop corrections (e.g., IBVS visual servo)
4. **Perception Updates:** Perception Module processes camera data to update world state
5. **Supervisor Monitoring:** Supervisor tracks task progress and detects anomalies

Lateral Communication

Between-module communication enables coordination:

- **Planning ↔ Perception:** Planning Module queries Perception Module for current object poses
- **Control ↔ Perception:** Control Module may request precision updates from Perception during delicate operations
- **Supervisor ↔ All Modules:** Supervisor broadcasts state information and receives status updates from all lower layers

2.1.8 Synchronization and Timing

Proper timing synchronization is critical for multi-robot coordination. The system employs:

- **ROS 2 Clock:** All nodes synchronized to ROS global clock (allows simulation time in Gazebo Harmonic)
- **Action Servers:** Asynchronous action servers ensure non-blocking execution of long-duration operations (trajectories, planning)
- **Watchdog Timers:** Detect stalled or crashed nodes through timeout mechanisms

2.1.9 Simulation Environment

The system is developed and tested using Gazebo Harmonic, a high-fidelity physics simulator:

- **Development Benefits:**
 - Test algorithms without risking hardware damage
 - Rapid prototyping and iteration
 - Deterministic simulation for algorithm validation

2.1.10 System Properties and Design Considerations

Modularity and Extensibility

The layered architecture enables:

- Independent development of each layer
- Swapping components (e.g., alternative planners, different gripper hardware) without system-wide refactoring
- Easy addition of new modules (e.g., force/torque feedback, additional sensors)

Robustness and Fault Tolerance

- **Graceful Degradation:** Loss of one capability triggers fallback strategies
- **Multi-Modal Control:** IBVS + Point Control provides redundancy
- **Recovery Logic:** Supervisor detects and recovers from common failures

Safety

- **Collision Detection:** Multiple layers check for collisions
- **Joint Limits:** All control layers respect hardware limits
- **Emergency Stop:** E-stop triggers immediate motion cessation at HAL layer

Scalability

The ROS 2 architecture scales to:

- Multiple robots (demonstrated with ABB + AR4)
- Additional sensors and actuators
- Distributed computation across multiple machines
- Future expansion to larger swarms of robots

2.2 AR4 Restoration and Hardware Modifications

The AR4 robotic manipulator was handed over to the current team by the previous graduation project team after their completion of the project. At the time of handover, the robot was delivered in a fully assembled condition and was generally operational. The unit is an AR4 MK3 model designed by Chris Annin. However, due to the duration and intensity of prior usage during development and testing, several components exhibited wear and assembly-related issues. As a result, multiple mechanical and electrical corrective actions were required before the robot could be considered reliable for accurate motion execution and experimental work.

2.2.1 Joint 3 Mechanical Transmission Upgrade

Joint 3 of the AR4 robot exhibited significant backlash due to wear and limitations in the original mechanical transmission design. To address this issue, the joint transmission system was completely replaced in accordance with the official upgrade provided by Chris Annin. The upgrade consisted of a redesigned gear set and a new timing belt, both intended to improve torque transmission and reduce mechanical play within the joint.

Figure 2.1: Upgraded mechanical transmission of Joint 3

After implementing the upgraded components, the backlash in Joint 3 was reduced from approximately 6° to 3° . This reduction resulted in smoother joint motion and improved positioning accuracy, directly enhancing the overall kinematic performance of the robot.

2.2.2 Shaft Key Re-Manufacturing in Joint 3

Despite the improvements achieved through the mechanical transmission upgrade, residual backlash was still observed in Joint 3. Further investigation revealed that the shaft key responsible for transmitting motion from the second gear stage to the robotic arm link exhibited wear and improper fit. This condition introduced mechanical play within the joint, negatively affecting positioning accuracy.

Figure 2.2: Re-manufactured shaft key used in Joint 3 to eliminate mechanical backlash

To resolve this issue, the shaft key was re-manufactured with precise dimensions to ensure a tight and accurate fit between the gear and the arm shaft using hot-wire machining. The newly manufactured key eliminated relative motion between the connected components, effectively removing the remaining backlash in Joint 3. As a result, Joint 3 achieved backlash-free motion.

2.2.3 Encoder Signal Noise in 5th and 6th joint

During system testing and motion execution, unstable and noisy encoder readings were observed in Joints 5 and 6 of the AR4 robot. These fluctuations resulted in inaccurate joint position feedback, which significantly affected the forward and inverse kinematics calculations. Consequently, commanded joint positions did not always correspond to the expected physical motion of the robot.

A systematic debugging process was conducted, including signal tracing and verification of the encoder wiring and power connections. This investigation revealed that the encoders of Joints 5 and 6 were not properly connected to the regulated 5 V power supply. The improper power connection caused voltage instability, leading to fluctuating encoder signals and unreliable position measurements.

After correcting the power supply connections and ensuring proper grounding, the encoder signals became stable and consistent. This correction restored accurate joint position feedback.

2.2.4 Joint 1 Backlash Due to Missing Fasteners

During mechanical inspection and testing of the AR4 robot, abnormal backlash was observed in Joint 1. This backlash was estimated to be approximately 2° and was found to adversely affect the base positioning accuracy of the robot, particularly during large workspace movements.

A detailed mechanical inspection revealed that multiple fastening screws within the Joint 1 assembly were missing. The absence of these fasteners reduced the structural rigidity of the joint, allowing relative motion between assembled components and introducing mechanical play during operation.

After identifying the missing screws, all required fasteners were reinstalled and properly tightened according to the recommended assembly specifications. This action restored the mechanical integrity of Joint 1 and eliminated the observed backlash. As a result, the positional accuracy and stability of the robot base joint were significantly improved.



2.2.5 Summary

Following the completion of all mechanical restorations and electrical corrections, the AR4 robot reached a stable and reliable hardware state suitable for precise operation. The corrective actions performed eliminated sources of mechanical backlash, improved structural rigidity, and ensured accurate encoder feedback across all joints. With these issues resolved, the robot was prepared for accurate kinematic modeling and motion execution.

Based on this finalized hardware condition, a comprehensive calibration procedure was subsequently carried out to align the mathematical kinematic model with the physical robot. The calibration methodology, implementation, and results are presented in the following section.

2.3 Cartesian Calibration of the AR4 Collaborative Robot

Accurate positioning is critical in collaborative robotic applications, particularly when multiple manipulators operate within a shared workspace. During initial experimental trials, the AR4 robotic arm exhibited noticeable Cartesian positioning inaccuracies when executing commanded end-effector positions relative to the ABB robot world coordinate frame. These inaccuracies motivated the implementation of a dedicated Cartesian calibration procedure to improve the absolute positioning accuracy of the AR4 robot.

2.3.1 Experimental Setup

The calibration experiment was conducted using a pointed end-effector mounted on the AR4 robot to enable precise position referencing. A calibrated measurement grid with millimeter resolution was placed on the worktable. The grid origin was aligned with the ABB robot world frame origin (0,0,0), ensuring a common global reference for both manipulators.

The AR4 robot was commanded to move to a predefined set of Cartesian positions distributed over the XY plane at multiple height levels along the Z-axis. At each commanded position ($x_{cmd}, y_{cmd}, z_{cmd}$), the actual end-effector position ($x_{meas}, y_{meas}, z_{meas}$) was manually recorded using the grid.

2.3.2 Cartesian Error Characterization

The raw Cartesian positioning errors were computed as:

$$\begin{aligned} e_x &= x_{meas} - x_{cmd} \\ e_y &= y_{meas} - y_{cmd} \\ e_z &= z_{meas} - z_{cmd} \end{aligned} \tag{2.1}$$

Initial analysis revealed systematic and spatially correlated errors across the workspace, indicating that the inaccuracies were not purely random but instead dependent on the commanded Cartesian position.

2.3.3 Calibration Method Selection

Several calibration approaches were evaluated prior to selecting the final method. Table 2.1 summarizes the comparison between candidate techniques. The calibration dataset was divided into independent training and testing subsets to ensure that the reported performance reflects generalization rather than overfitting to the calibration points.

Method	Accuracy	Extrapolation	Data Requirements	Real-Time
Lookup Table	Medium	None	High	High
RBF Network	High	Poor	Very High	Medium
Spline Fitting	High	Limited	Medium	High
Polynomial Regression	High	Good (Low-Order)	Medium	High

Table 2.1: Comparison of Cartesian calibration methods for robotic manipulators

Based on this comparison, the polynomial calibration was selected as a balanced solution, offering a favorable trade-off between accuracy, robustness, and implementation simplicity, making it particularly suitable for real-time correction in collaborative robotic systems.

2.3.4 Polynomial Calibration Model

A second-order polynomial regression model was used to approximate the Cartesian error as a function of the commanded XY position:

$$e(x, y) = a_1x + a_2y + a_3x^2 + a_4y^2 + a_5xy + a_6 \quad (2.2)$$

The estimated polynomial error models were used to correct the commanded positions prior to execution:

$$\begin{aligned} x_{corr} &= x_{cmd} + \hat{e}_x(x, y) \\ y_{corr} &= y_{cmd} + \hat{e}_y(x, y) \\ z_{corr} &= z_{cmd} + \hat{e}_z(x, y) \end{aligned} \quad (2.3)$$

This correction strategy enables feedforward compensation of systematic positioning errors without modifying the internal robot controller.

2.3.5 Calibration Results

Based on the experimental results, all evaluated calibration approaches significantly reduced the Cartesian positioning error when compared to the uncalibrated model. Among the tested methods, the linear model exhibited the highest residual error due to its inability to capture nonlinear kinematic and geometric effects. Radial Basis Function (RBF) networks achieved the lowest mean positioning error, demonstrating superior interpolation accuracy within the calibrated workspace. However, their locally adaptive nature resulted in less predictable behavior near the workspace boundaries and outside the training region.

The second-degree polynomial model produced slightly higher errors than the RBF-based approach, yet these errors remained within acceptable limits for the intended application. In contrast to RBF and spline-based methods, the polynomial model exhibited smoother and more stable behavior near

workspace boundaries, while maintaining low computational cost and consistent real-time performance.

Figure 2.3 illustrates a comparison between the commanded, measured, and polynomial-corrected end-effector positions in the XY plane. The deviation between the commanded and measured positions highlights the presence of systematic Cartesian positioning errors in the AR4 robot, while the corrected positions demonstrate the effectiveness of the polynomial calibration in compensating for these errors.

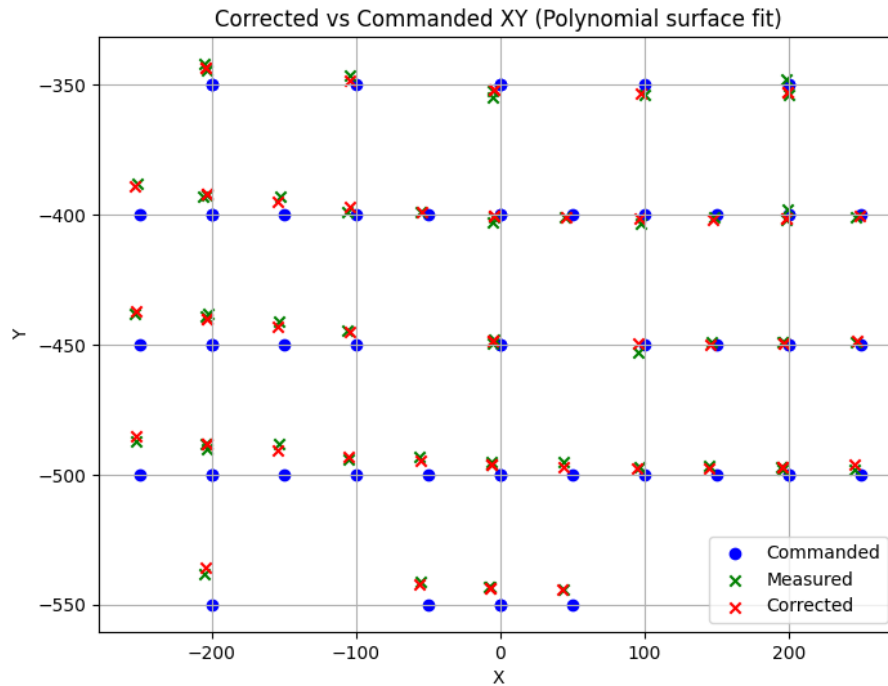


Figure 2.3: XY-plane commanded, measured, and corrected positions.

2.3.6 Error Surface Analysis

To further analyze the spatial behavior of the calibration model, three-dimensional polynomial error surfaces were generated for each Cartesian axis, which are shown in Figure 2.4

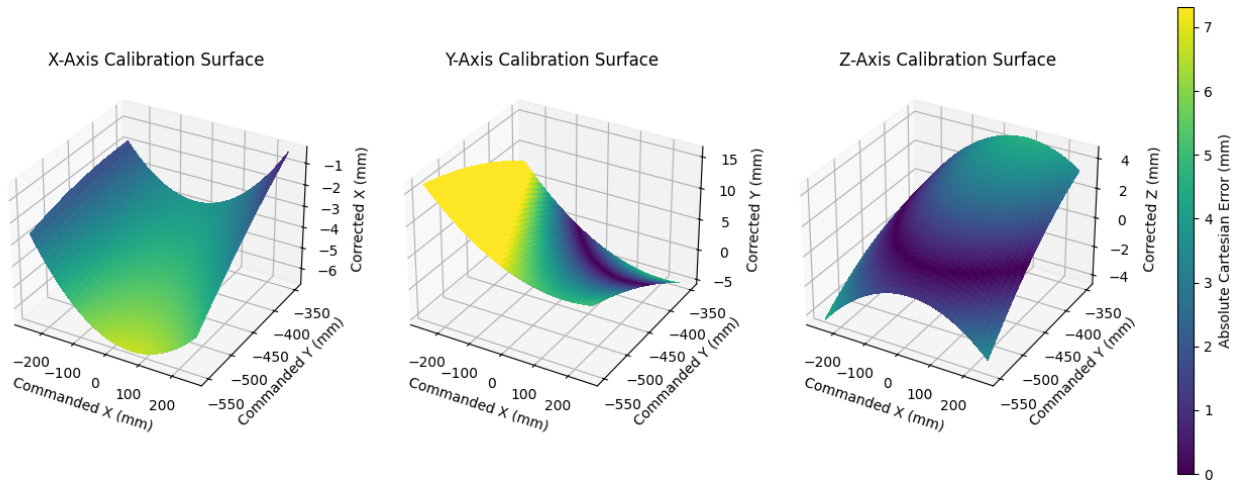


Figure 2.4: Calibration surfaces for the X, Y, and Z axes with error magnitude.

Figure 2.5 presents a comparison between the uncalibrated and calibrated surfaces. The comparison demonstrates that the calibration model effectively smooths and compensates for the dominant systematic error components, resulting in a more uniform and stable error distribution across the workspace.

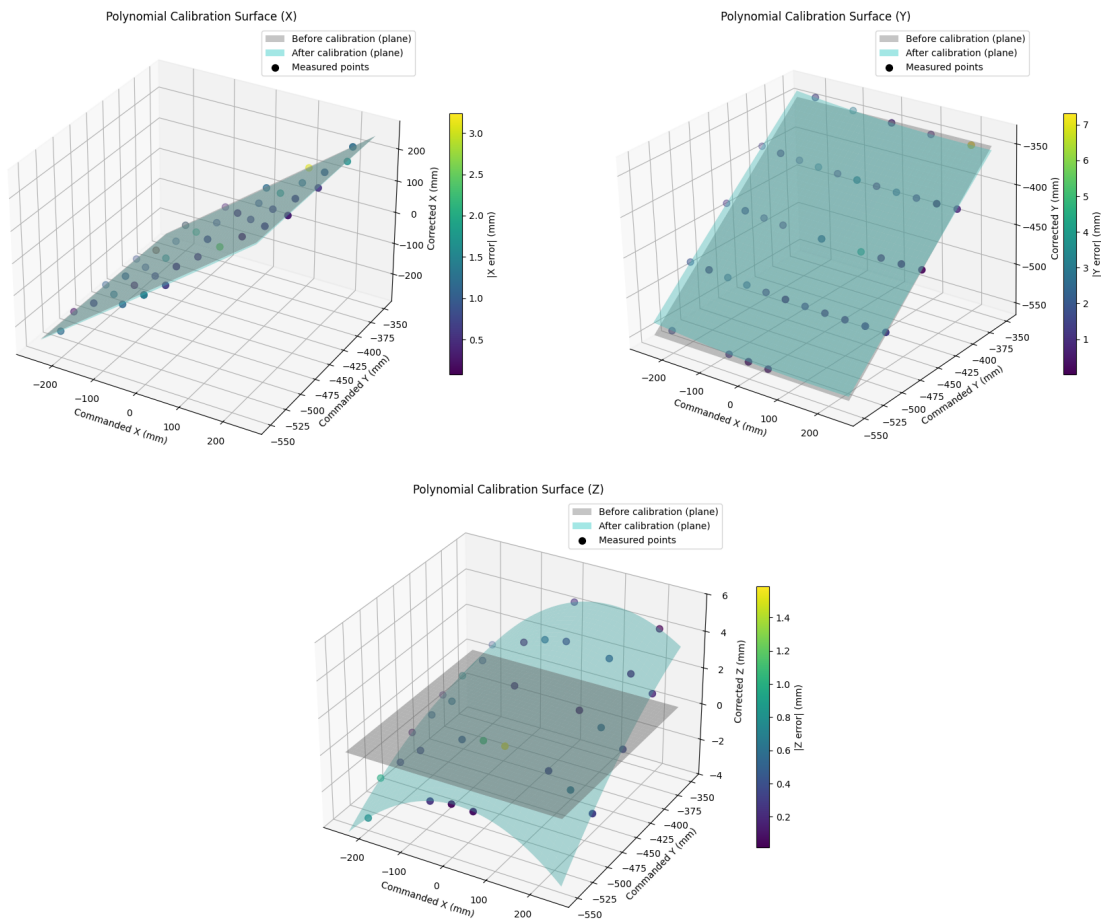


Figure 2.5: Comparison of uncalibrated and calibrated surfaces.

2.3.7 Quantitative Evaluation

The effectiveness of the calibration was quantitatively evaluated using the mean absolute error (MAE) before and after correction for each Cartesian axis, computed on the independent test dataset.

Table 2.2: Mean Absolute Cartesian Errors Before and After Calibration

Axis	MAE Before (mm)	MAE After (mm)
X	3.96	0.94
Y	5.13	1.32
Z	1.94	0.46

The quantitative results in Table 2.2 are consistent with the smooth and continuous behavior observed in the polynomial error surfaces shown in Figure 2.4, confirming the effectiveness and stability of the proposed calibration approach.

2.4 Assembly Supervisor and State Machine Control

2.4.1 Overview

The Assembly Supervisor is the central orchestrator of the collaborative robotic assembly system. It implements a robust state machine that governs the complete workflow from task initialization through brick detection, grasp planning, robot coordination, and final assembly placement. This section provides a comprehensive analysis of the supervisor's architecture, state transitions, service and action interactions, and error handling mechanisms that ensure reliable system operation in a collaborative multi-robot environment.

2.4.2 Supervisor Architecture and Responsibilities

High-Level Architecture

The Assembly Supervisor operates as a ROS 2 node with the following core responsibilities:

- **Task Coordination:** Retrieve assembly plans from the GUI and manage task allocation across the two robotic arms.
- **Perception Integration:** Coordinate camera-based brick detection and transform detected poses into the appropriate reference frames.
- **Grasp Inference:** Query the grasp pipeline to select stable grasps for each brick.
- **Motion Execution:** Issue action goals to the AR4 and ABB control servers to execute pick, place, and handover operations.
- **Coordination Logic:** Decide between direct assembly and handover strategies based on brick location and arm allocation.
- **Reference Frame Management:** Transform all detected poses and grasp points into the appropriate coordinate frames using TF2 (ROS 2 Transform Library).

Execution Model and Threading

The supervisor employs a multi-threaded executor with a `ReentrantCallbackGroup` to permit concurrent handling of asynchronous ROS 2 callbacks. This design allows:

- Parallel service queries and action goal submissions without blocking the main state machine loop.
- Safe re-entrant locking when multiple callbacks access shared state variables.
- Non-blocking timer-based state machine updates at a controlled frequency (initially 1.0 Hz, reduced to 0.1 Hz during active task execution).

Figure 2.6: Assembly Supervisor Architecture: Clients, Callbacks, and State Machine

2.4.3 ROS 2 Communication: Services and Actions

Service Clients

Services provide synchronous request-response communication for deterministic operations that do not require real-time feedback:

1. `get_assembly_plan` (Service)

- **Purpose:** Retrieve the assembly plan (ordered list of bricks) from the GUI node.
- **Interface:** `supervisor_package/srv/GetAssemblyPlan`
- **Response Fields:** Plan array containing brick ID, location, pickup pose, and place pose.
- **Invocation:** Called once during INIT state to bootstrap the assembly sequence.

2. `detect_bricks` (Service)

- **Purpose:** Trigger YOLOv8 segmentation and brick pose estimation from the global RealSense camera.
- **Interface:** `dual_arms_msgs/srv/DetectBricks`
- **Response Fields:** Array of detected bricks with world poses, plus a computed handover pose (neutral zone between robots).
- **Invocation:** Called during DETECT state to refresh spatial understanding after assembly plan receipt.

3. `grasp/get_grasp_point` (Service)

- **Purpose:** Query the transformer-based grasp inference model for the optimal grasp candidate for a specific brick.
- **Interface:** `dual_arms_msgs/srv/GetGrasp`
- **Request Fields:** Brick index (converted to string).
- **Response Fields:** A `GraspPoint` message containing pose, quality metric, and success indicator.
- **Invocation:** Called for each brick during GRASP_PIPELINE state before attempting to grasp.

Action Clients

Actions provide asynchronous, feedback-capable communication for long-running operations. The supervisor uses action clients to submit motion goals and wait for completion:

1. `ar4_point_control` (Action)

- **Purpose:** Send point-to-point motion commands to the AR4 arm (approach, grasp, release, placement).
- **Interface:** `supervisor_package/action/MoveToPose`
- **Goal Fields:** Target pose, strategy (APPROACH_OFFSET, GRASP, PLACE, GOTO_HANDOVER, RELEASE).
- **Used in States:** EXECUTE_AR4_DIRECT, AR4_PICK_FOR_HANDOVER, AR4_PLACE_ON_GRID, HANDOVER_EXECUTION.

2. ar4_visual_servo (Action)

- **Purpose:** Execute Image-Based Visual Servoing (IBVS) feedback control via the ESP32-CAM.
- **Interface:** `supervisor_package/action/AlignToTarget`
- **Goal Fields:** Object ID (brick type), homography mapping parameters.
- **Behavior:** Iteratively refines AR4 position to align the brick in the ESP32 camera frame, halting once pixel error is below threshold.
- **Used in States:** EXECUTE_AR4_DIRECT, AR4_PICK_FOR_HANDOVER.

3. abb_control (Action)

- **Purpose:** Issue high-level task commands to the ABB robot (pick from detected pose, place, pick from handover).
- **Interface:** `dual_arms_msgs/action/ExecuteTask`
- **Goal Fields:** Task type (PICK, PLACE, PICK_FROM_HANDOVER), target pose.
- **Behavior:** The ABB controller internally manages gripper open/close, motion planning, and collision avoidance.
- **Used in States:** EXECUTE_ABB_PICK, EXECUTE_ABB_PLACE, HANDOVER_EXECUTION.

Communication Flow Diagram

Figure 2.7: Supervisor Service and Action Communication Flow

2.4.4 Transform (TF2) Management and Coordinate Frames

Coordinate Frame Hierarchy

The system employs multiple coordinate frames, necessitating careful transformation management:

- **base_link:** The ABB robot base (designated as the primary world reference frame).
- **camera_color_optical_frame:** The RealSense camera's optical center.
- **ar4_base:** The AR4 robot's base frame.

- **ar4_tool:** The AR4 end-effector (gripper) frame.
- **esp32_cam_optical:** The ESP32-CAM's optical frame (eye-in-hand).
- **table:** A virtual frame representing the assembly table surface.

Transform Pipeline

The supervisor uses TF2 to establish and maintain transformations:

1. Brick Detection to World Frame:

- Input: Brick pose in `camera_color_optical_frame` from YOLOv8 segmentation.
- Transformation: `lookup_transform('base_link', 'camera_color_optical_frame')`
- Output: Brick pose in `base_link` (ABB world frame) for unified task planning.

2. Grasp Point Transformation:

- The grasp pipeline returns a `GraspPoint` in the camera frame.
- The supervisor applies the same camera-to-base transformation.
- Additional adjustments are made: Z-coordinate clamped to safety height (0.22 m), orientation components reordered to match AR4 conventions.

3. Handover Pose Transformation:

- The global detection service computes a handover pose (midpoint between the two robots).
- The supervisor transforms this pose to `base_link` for consistency.

Transform Implementation Details

The supervisor implements transform lookups with error handling. A typical lookup operation:

```
def transform_pose_to_abb(self, input_pose):
    try:
        t = self.tf_buffer.lookup_transform(
            'base_link',
            'camera_color_optical_frame',
            rclpy.time.Time())
        transformed_pose = tf2_geometry_msgs.do_transform_pose(
            input_pose, t)
        return transformed_pose
    except TransformException as ex:
        self.get_logger().error(f'TF2 Error: {ex}')
        return input_pose # Fallback
```

This implementation:

- Uses `rcipy.time.Time()` to request the latest available transform.
- Falls back to the original pose if transformation fails (avoiding system crash).
- Logs transformation errors for debugging.

Figure 2.8: Coordinate Frame Hierarchy and Transform Chain

2.4.5 Detailed State Machine Analysis

State Machine Overview

The Assembly Supervisor implements a 12-state finite state machine that governs the complete assembly workflow. States are organized into five logical stages:

1. **Stage 1 - Initialization (INIT):** Retrieve assembly plan and validate input.
2. **Stage 2 - Perception (DETECT):** Obtain global brick poses and compute handover zone.
3. **Stage 3 - Task Allocation (PROCESS_NEXT, GRASP_PIPELINE):** Select next brick and compute grasp.
4. **Stage 4 - Execution (Multiple states):** Execute grasping, handover, or placement based on allocation.
5. **Stage 5 - Termination (DONE):** All tasks complete; system idle.

Figure 2.9: Complete State Machine Diagram with Transitions

State 1: INIT (Initialization)

Purpose Retrieve the assembly plan from the GUI and bootstrap the state machine. This state validates that a plan is available before proceeding to perception.

Execution Logic

1. Check if the `gui_client` service is available using `wait_for_service(timeout_sec=1.0)`.
2. If unavailable, log a message and reschedule the state machine loop after a 2-second delay.
3. If available, submit a `GetAssemblyPlan` service request (empty request body).
4. Validate the response: Check that `result.plan` is not `None` and contains at least one brick.
5. On valid plan receipt, store plan in `self.assembly_queue` and transition to DETECT.
6. On invalid plan, log a warning and reschedule after 2 seconds (remain in INIT).

Error Handling

- **Service Not Ready:** Reschedule and wait. This handles GUI node startup delays.
- **Empty Plan:** Log and retry. This distinguishes between “service not ready” and “no plan available”.
- **Network Failure:** Timeout on `wait_for_service` is caught; state remains INIT.

Transition

- Success: INIT → DETECT
- Retry: INIT → INIT (after 2 seconds)

State 2: DETECT (Global Perception)

Purpose Invoke YOLOv8 segmentation and pose estimation to obtain brick positions in the global camera frame. Transform all poses to the ABB base frame.

Execution Logic

1. Check service availability: `self.camera_client.wait_for_service(timeout_sec=1.0)`.
2. If unavailable, log error and reschedule.
3. Submit a `DetectBricks` service request.
4. Receive response containing:
 - `result.bricks[*]`: Array of detected bricks with pose in camera frame.
 - `result.handover_pose`: Computed neutral zone pose in camera frame.
5. For each brick, apply transform: `brick.pose = self.transform_pose_to_abb(brick.pose)`.
6. Similarly, transform handover pose: `self.handover_pose = self.transform_pose_to_abb(result.handover_pose)`.
7. Store transformed bricks in `self.detected_bricks`.
8. Transition to `PROCESS_NEXT`.

Pose Validation After transformation, poses are validated:

- Z-coordinate should be non-negative (on or above table).
- X, Y coordinates should be within workspace bounds.
- Orientation quaternion should be normalized.

Transition

- Success: DETECT → PROCESS_NEXT

State 3: PROCESS_NEXT (Task Dequeue)

Purpose Remove the next brick from the assembly queue and initiate grasp planning. This state acts as a checkpoint for per-brick initialization.

Execution Logic

1. Check if `self.assembly_queue` is empty.
2. If empty: transition to DONE and exit.
3. If non-empty: pop the first element and assign to `self.current_brick`.
4. Transition to GRASP_PIPELINE.

Transition

- Bricks remaining: PROCESS_NEXT → GRASP_PIPELINE
- No bricks: PROCESS_NEXT → DONE

State 4: GRASP_PIPELINE (Grasp Inference)

Purpose Query the transformer-based grasp inference model to obtain a stable grasp candidate for the current brick. Apply pose transformation and additional safety adjustments.

Execution Logic

1. Check service availability: `self.grasp_pipeline_client.wait_for_service(timeout_sec=2.0)`.
2. If unavailable, log error, transition to PROCESS_NEXT (skip brick), and return.
3. Create a `GetGrasp.Request()` with `brick_index = str(self.current_brick.id)`.
4. Submit request asynchronously and await response.
5. Check `grasp_result.success` flag:
 - If True: Extract `grasp_result.grasp_point` (a `GraspPoint` message).
 - If False: Log error, transition to PROCESS_NEXT, skip brick.

6. Apply coordinate frame transformation.
7. Apply safety adjustments (clamp Z-coordinate, reorder orientation).
8. Store transformed grasp in `self.current_grasp_point`.
9. Determine arm allocation based on `self.current_brick.location`.

Grasp Point Structure The `GraspPoint` message contains:

<code>geometry_msgs/Pose pose</code>	# Grasp position and orientation
<code>float32 quality</code>	# Quality metric [0, 1]
<code>float32 width</code>	# Finger opening distance (meters)
<code>float32 approach_angle</code>	# Recommended approach angle (radians)

Arm Allocation Logic Based on brick location from the assembly plan:

- `location == 1`: Brick on ABB's side → `EXECUTE_ABB_PICK`
- `location == 2`: Brick requires handover → `HANDOVER_SEQUENCE`
- Other: Brick on AR4's side → `EXECUTE_AR4_DIRECT`

Transition

- Grasp success, ABB-side brick: `GRASP_PIPELINE` → `EXECUTE_ABB_PICK`
- Grasp success, AR4-side brick: `GRASP_PIPELINE` → `EXECUTE_AR4_DIRECT`
- Grasp success, handover-required: `GRASP_PIPELINE` → `HANDOVER_SEQUENCE`
- Grasp failure: `GRASP_PIPELINE` → `PROCESS_NEXT`

State 5: `EXECUTE_AR4_DIRECT` (AR4 Pick and Place)

Purpose Execute a complete pick-and-place cycle using the AR4 robot for bricks allocated to the AR4's workspace.

Sub-Step A: Approach with Offset

1. Create a `MoveToPose` action goal with strategy "`APPROACH_OFFSET`".
2. The AR4 controller offsets in the approach direction to maintain safe clearance.
3. Await completion.

Sub-Step B: Visual Servoing (IBVS)

1. Create an `AlignToTarget` action goal with object ID and homography parameters.
2. The visual servoing action server:
 - Captures images from the ESP32-CAM.
 - Detects the brick and computes image-space error.
 - Uses IBVS feedback law to generate AR4 velocity commands.
 - Continues until error is below threshold.
3. Await completion.

Sub-Step C: Grasp and Retract

1. Create a `MoveToPose` action goal with strategy "GRASP".
2. The AR4 controller executes final approach and closes the gripper.
3. Await completion.

Transition

- Success: `EXECUTE_AR4_DIRECT` → `AR4_PLACE_ON_GRID`

State 6: `AR4_PLACE_ON_GRID` (AR4 Placement)

Purpose Move the grasped brick to the assembly grid location specified by the plan.

Execution Logic

1. Create a `MoveToPose` action goal with strategy "PLACE".
2. Submit to AR4 point control action server.
3. The AR4 controller moves and opens the gripper.
4. Await completion.
5. Transition to `PROCESS_NEXT`.

Transition

- Success: `AR4_PLACE_ON_GRID` → `PROCESS_NEXT`

State 7: EXECUTE_ABB_PICK (ABB Grasping)

Purpose Execute pick operation using the ABB robot for bricks on the ABB's side of the workspace.

Execution Logic

1. Create an `ExecuteTask` action goal with task type "PICK".
2. Submit to ABB control action server.
3. The ABB controller plans and executes the pick sequence.
4. Await completion.
5. Transition to EXECUTE_ABB_PLACE.

Transition

- Success: EXECUTE_ABB_PICK → EXECUTE_ABB_PLACE

State 8: EXECUTE_ABB_PLACE (ABB Placement)

Purpose Place the grasped brick at its designated location on the assembly grid.

Execution Logic

1. Create an `ExecuteTask` action goal with task type "PLACE".
2. Apply safety adjustment: Set Z-coordinate to 0.24 m.
3. Submit to ABB control action server.
4. Await completion.
5. Transition to PROCESS_NEXT.

Transition

- Success: EXECUTE_ABB_PLACE → PROCESS_NEXT

State 9: HANDOVER_SEQUENCE (Handover Initiation)

Purpose Transition to handover workflow when a brick must be transferred from AR4 to ABB.

Execution Logic

1. This state is a pass-through (no action).
2. Immediately transition to AR4_PICK_FOR_HANDOVER.

Transition

- HANDOVER_SEQUENCE → AR4_PICK_FOR_HANDOVER

State 10: AR4_PICK_FOR_HANDOVER (AR4 Grasping for Transfer)

Purpose Execute the AR4 grasping sequence in preparation for handover.

Execution Logic See Sub-Steps A, B, C under State 5 (EXECUTE_AR4_DIRECT).

Transition

- Success: AR4_PICK_FOR_HANDOVER → HANDOVER_EXECUTION

State 11: HANDOVER_EXECUTION (Coordinated Transfer)

Purpose Execute synchronized handover between the AR4 and ABB robots.

Step 1: AR4 Move to Handover Zone

1. Create a MoveToPose action goal with strategy "GOTO_HANDOVER".
2. Submit to AR4 point control action server.
3. Await completion.

Step 2: ABB Approach and Grasp

1. Create an ExecuteTask action goal with task type "PICK_FROM_HANDOVER".
2. Submit to ABB control action server.
3. The ABB controller approaches and closes gripper.
4. Await completion.

Step 3: AR4 Release

1. Create a `MoveToPose` action goal with strategy "RELEASE".
2. Submit to AR4 point control action server.
3. The AR4 controller opens gripper and retracts.
4. Await completion.

Transition

- Success: `HANDOVER_EXECUTION` → `PROCESS_NEXT`

Handover Synchronization Considerations

- **Sequential Approach:** Strict sequential steps ensure clear state and avoid conflicts.
- **Force Feedback (Future):** Force/torque sensors could enable simultaneous dual-robot grasp confirmation.
- **Fallback Strategy:** If ABB grasp fails, AR4 does not release; system can retry.

State 12: DONE (Completion)

Purpose Signal that all tasks are completed and the system is idle.

Execution Logic

1. Log completion message.
2. Do not reschedule the state machine loop timer.
3. System waits for external INIT signal or restart.

Transition

- `DONE` → (Await external signal)

2.4.6 Error Handling and Recovery

Service Availability Checking

The supervisor checks service availability before submitting requests. If unavailable:

- Log a diagnostic message.
- Reschedule the state machine loop after a delay (2 seconds).
- Remain in current state, retrying the same action.

Action Goal Failure

When an action goal is rejected:

- Check the goal handle status.
- Log an error and return False.
- The calling state decides on recovery (retry or skip).

Transform Lookup Failures

If a TF2 transform is unavailable:

- Catch `TransformException`.
- Log the exception.
- Return the original pose as fallback.

Grasp Pipeline Failures

If the grasp service returns `success=False`:

- Log the failure.
- Transition to `PROCESS_NEXT` (skip the brick).

Robot Timeout

If an action goal does not complete within a reasonable time:

- Cancel the goal and log the failure.
- Transition to `PROCESS_NEXT`.
- Proceed with the next brick.

2.4.7 Concurrency and Threading Model

ReentrantCallbackGroup

The supervisor uses `ReentrantCallbackGroup` to allow concurrent callback execution:

- Asynchronous service responses processed during state machine operation.
- Non-blocking behavior for long-running actions.

MultiThreadedExecutor

The supervisor node uses `MultiThreadedExecutor`:

- Allocates thread pool for callback handling.
- Async/await patterns execute without blocking.
- Safe re-entrant locking for shared state.

Potential Race Conditions

- **assembly_queue modification:** Mitigated by single-threaded state machine logic.
- **current_brick and grasp data:** Protected by sequential state transitions.
- **detected_bricks refresh:** Snapshot consistency assumed.

2.4.8 State Machine Timing and Scheduling

Timer Management

The supervisor uses `rclpy.time.Timer` for periodic state machine invocations:

- **Initialization:** Timer created at 1.0 Hz frequency.
- **Dynamic Frequency:** Reset to 0.1 Hz during heavy computation to reduce CPU overhead.
- **Cancellation:** Timer canceled when state machine completes or encounters blocking actions.

Async/Await Pattern

The state machine loop is asynchronous, enabling:

- Non-blocking waiting for service responses.
- Clean exception handling and state transitions.
- Delayed task scheduling and timer rescheduling.

2.4.9 Hardware and Sensor Integration

Robotic Arms

ABB Robot

- **Model:** IRB 1200-5/90 (industrial manipulator).
- **Workspace:** Covers placement side of assembly table.
- **Gripper:** Two-finger parallel gripper.
- **Communication:** ROS 2 action server at `abb_control`.
- **Payload Capacity:** Sufficient for single brick grasping.

AR4 Robot

- **Model:** Annin Robotics AR4 (4-DOF SCARA-style arm).
- **Workspace:** Covers grasping/pickup side.
- **Gripper:** Two-finger gripper controlled via ESP32.
- **Eye-in-Hand Camera:** ESP32-CAM for local perception.
- **Communication:** ROS 2 action servers at `ar4_point_control` and `ar4_visual_servo`.
- **Calibration:** Position correction via polynomial regression (Section 2.3).

Cameras and Perception

Intel RealSense D435i (Global Perception)

- **Placement:** Fixed above assembly table covering full workspace.
- **Output:** RGB images for YOLOv8 segmentation.
- **Calibration:** Camera-to-ABB-base transformation in TF2.
- **Update Rate:** 30 Hz.

ESP32-CAM (Eye-in-Hand Perception)

- **Placement:** Mounted on AR4 end-effector.
- **Output:** Low-resolution RGB for feature detection and visual servoing.
- **Calibration:** Homography mapping from RealSense to ESP32 view.
- **Update Rate:** 5–10 Hz.
- **Latency:** 100–200 ms (mitigated by low-gain feedback control).

2.4.10 Data Flow and Message Passing

Brick Representation

The system represents bricks with the following structure:

```
dual_arms_msgs/Brick:
  int32 id                # Unique identifier
  int32 type              # Brick class
  int32 location          # 1=ABB side, 2=Handover, other=AR4
  geometry_msgs/Pose pickup_pose
  geometry_msgs/Pose place_pose
```

Grasp Point Representation

Grasp candidates are represented as:

```
dual_arms_msgs/GraspPoint:
  geometry_msgs/Pose pose    # Position and orientation
  float32 quality            # Metric [0, 1]
  float32 width              # Finger opening [meters]
  float32 approach_angle     # Approach direction
```

2.4.11 Debugging and Logging

Log Levels

The supervisor uses ROS 2 logging with different verbosity levels:

- **INFO:** State transitions, service calls, task completion.
- **WARN:** Empty plans, service delays, non-critical failures.
- **ERROR:** Service unavailability, transform failures, action rejections.

State Machine Monitoring

Current logging provides visibility into:

- State transitions.
- Task counts.
- Brick-specific progress.
- Grasp quality metrics.

2.4.12 Summary

The Assembly Supervisor implements a robust, multi-threaded state machine that orchestrates a collaborative two-robot assembly system. Key features include:

1. **Modular Service/Action Architecture:** Clean separation between perception, execution, and decision logic.
2. **Transform Management:** Consistent use of TF2 for coordinate frame conversions.
3. **Comprehensive State Machine:** 12 states covering all operational phases with clear error recovery paths.
4. **Concurrency Support:** MultiThreadedExecutor enables responsive, non-blocking operation.
5. **Hardware Integration:** Direct control of two heterogeneous arms with different control models.
6. **Error Resilience:** Timeouts, service failures, and inference failures handled gracefully.

The supervisor's structured approach provides a foundation for reliable, repeatable collaborative assembly operations and facilitates integration with the Digital Twin for monitoring and optimization.

2.5 Introduction

In collaborative robotic tasks, achieving high-precision alignment for grasping objects is a significant challenge, particularly when the environment and arm cameras have different perspectives. This chapter details the development of a "Fine Control" layer for the AR4 robotic arm using Image-Based Visual Servoing (IBVS) and Homography-based mapping. Our approach utilizes a dual-camera system: a fixed global RealSense camera for initial brick detection and a local ESP32-CAM mounted on the end-effector for precise alignment. By bridging these two views with homography, we enable the robot to perform complex "Pick and Place" operations with high reliability and reduced computational overhead.

2.6 Literature Review

The development of this module was informed by several key research areas in robotic vision and control, focusing on robustness, multi-camera coordination, and non-conventional optics.

2.6.1 Visual Servo Control Foundations

Visual servoing is a robotic control technique that utilizes information from a vision sensor to regulate the pose (position and orientation) of a robot's end-effector relative to a target or environmental features [1]. It represents the fusion of results from multiple disciplines, including high-speed image processing, kinematics, dynamics, control theory, and real-time computing [1]. The fundamental objective is to create a closed-loop control system where visual information is used to minimize a vision-based error signal.

2.6.2 The Pinhole Camera Model

The foundation of visual servoing rests on the geometric mapping of 3D points in the environment onto a 2D image plane. As illustrated in Figure 2.10, this process is modeled using the pinhole camera projection [1].

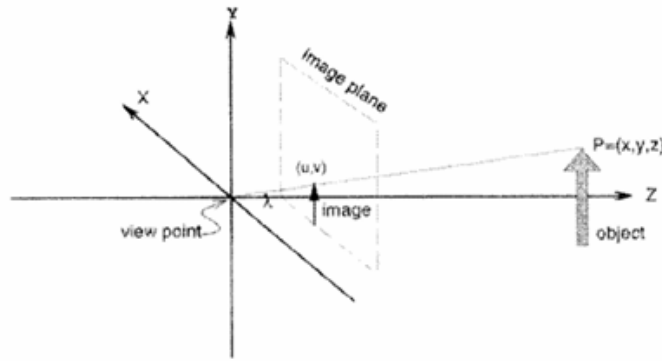


Figure 2.10: Perspective projection geometry showing the relationship between a 3D point P and its 2D image projection p [1].

For a 3D point $P = [X, Y, Z]^T$ in the camera frame $\{C\}$, its projection onto the image plane at coordinates $p = [x, y]^T$ is determined by the focal length λ through similar triangles [1]:

$$x = \lambda \frac{X}{Z}, \quad y = \lambda \frac{Y}{Z} \quad (2.4)$$

This relationship is critical because it forms the starting point for the derivation of the **Interaction Matrix** (Image Jacobian). By differentiating these coordinates with respect to time, we can relate the velocity of image features $\dot{s}$ to the spatial velocity of the camera v_c [1].

2.6.3 The Pinhole Camera Model

The foundation of visual servoing rests on the geometric mapping of 3D points onto a 2D image plane. This process is modeled using the pinhole camera projection, where a 3D point $P = [X, Y, Z]^T$ in the camera frame $\{C\}$ is projected onto the image plane at coordinates $p = [x, y]^T$ via the focal length λ [1]:

$$x = \lambda \frac{X}{Z}, \quad y = \lambda \frac{Y}{Z} \quad (2.5)$$

This projection is critical as it provides the geometric basis for the **Interaction Matrix** (Image Jacobian) used in the control law [1].

2.6.4 Position-Based Visual Servo (PBVS)

In PBVS architectures, features extracted from image data are used in conjunction with a known geometric model of the target and camera parameters to estimate the target's pose relative to the camera frame [1].

Kinematic Error Function and Control Law

A positioning task in PBVS is represented by a kinematic error function $E : \mathcal{T} \rightarrow \mathbb{R}^m$, where $\mathcal{T}$ is the task space of the robot [1]. The task is fulfilled when the end-effector is in a pose x_e such that $E(x_e) = 0$. The regulator typically computes a desired end-effector velocity screw u via:

$$u = -k\hat{E}(x_e) \quad (2.6)$$

where $k > 0$ is a proportional feedback gain and $\hat{E}$ is the estimated error based on reconstructed 3D geometry. The primary challenge remains the visual reconstruction problem, which is highly sensitive to camera calibration and robot kinematics.

2.6.5 Image-Based Visual Servo (IBVS)

IBVS defines the control task directly in the image feature space $\mathcal{F}$, avoiding the need for a full 3D interpretation of the scene [1]. The error signal is defined as $e(t) = f(t) - f_d$, where f represents the current vector of image feature parameters and f_d represents the desired feature configuration [1,3].

The Image Jacobian (Interaction Matrix)

The Image Jacobian J_v is a linear transformation relating the rate of change in image feature parameters $\dot{f}$ to the camera's spatial velocity screw $\dot{r}$:

$$\dot{f} = J_v(r)\dot{r} \quad (2.7)$$

[Image of an Image Jacobian matrix equation]

For a 3D point $P = [x, y, z]^T$ under perspective projection with focal length λ , the velocity of its image plane coordinates $[u, v]^T$ relative to camera translational velocity T and angular velocity Ω is:

$$\begin{bmatrix} \dot{u} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} \frac{\lambda}{z} & 0 & -\frac{u}{z} & -\frac{uv}{\lambda} & \frac{\lambda^2+u^2}{\lambda} & -v \\ 0 & \frac{\lambda}{z} & -\frac{v}{z} & \frac{-\lambda^2-v^2}{\lambda} & \frac{uv}{\lambda} & u \end{bmatrix} \begin{bmatrix} T_x \\ T_y \\ T_z \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad (2.8)$$

The manipulator velocity u is then computed using the Moore-Penrose pseudoinverse J_v^+ :

$$u = -KJ_v^+(r)e(f) \quad (2.9)$$

Advanced IBVS and Refined Frameworks

Modern implementations utilize image moments m_{ij} to describe complex shapes effectively, often combined with a **Coarse-to-Fine** framework for sub-millimeter accuracy [3,4]. This involves removing large initial alignment errors (15–30 cm) before using fine controllers to refine the pose within a 0.6 mm tolerance.

2.6.6 Comparative Analysis: IBVS vs. PBVS

The two primary paradigms in visual servoing differ in how they define the error signal and the space in which control is executed [1].

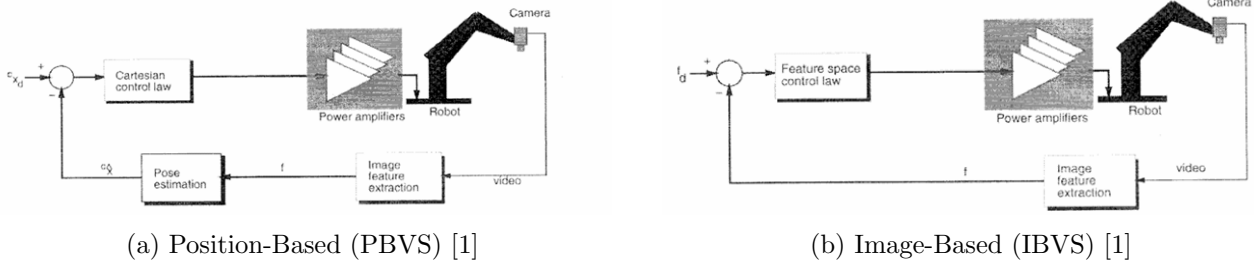


Figure 2.11: Control loop architectures. PBVS (left) relies on a 3D pose estimation block, while IBVS (right) regulates error directly in the image plane [1].

As illustrated in Figure 2.11, PBVS is highly dependent on accurate 3D reconstruction, which introduces sensitivity to camera calibration errors [1]. In contrast, the IBVS approach regulates features directly in the image plane, making it inherently more robust to kinematic uncertainties and calibration biases [1].

The selection of a visual servoing architecture involves critical trade-offs:

- **Calibration Sensitivity:** IBVS is inherently less sensitive to camera calibration and robot kinematic errors compared to PBVS [1].
- **Computational Complexity:** IBVS avoids nonlinear optimizations for 3D pose estimation but requires online computation of the depth-dependent Jacobian.
- **Stability:** IBVS can encounter singularities in the feature mapping function, while PBVS remains more intuitive for global stability in the Cartesian task space.

2.6.7 Homography-Based Euclidean Reconstruction

A significant advancement in bridging the gap between 2D features and 3D control is the use of Homography-based visual servoing. As established by Fang et al., a homography matrix H relates the image coordinates of a target in a current view (m_i) to a desired reference view (m_i^*) through the relationship [2]:

$$m_i = H m_i^* \quad (2.10)$$

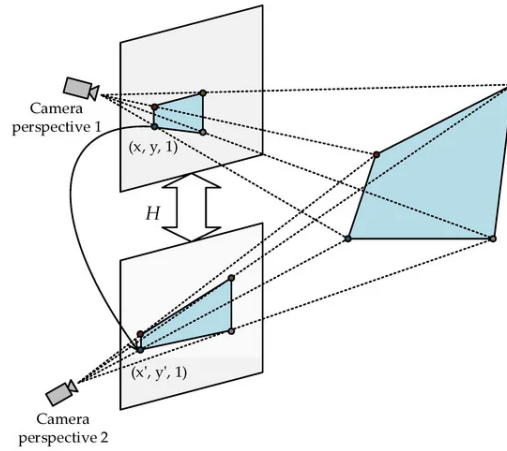


Figure 2.12: Schematic diagram of the planar homography transformation relationship between current and desired camera frames. [5]

The homography matrix can be decomposed to achieve a Euclidean reconstruction of the scene. Specifically, H is composed of the rotation R and the translation t between the two camera views, the normal vector n^* of the object plane, and the distance d^* from the reference camera to the plane [2]:

$$H = R + \frac{t}{d^*} n^{*T} \quad (2.11)$$

This approach allows the system to recover the position and orientation of the robot relative to the target without requiring an explicit 3D model. In multi-camera systems, this enables the automation paradigm to shift from point-centric to scene-centric, effectively mapping coordinates observed in a global view (e.g., RealSense) into a local view (e.g., ESP32-CAM) to maintain asymptotic regulation of the end-effector [2, 4].

2.7 General Concept and Control Flow

Visual servoing (VS) is a sensor-based control technique that utilizes real-time vision data as feedback to control the motion of a robot manipulator. In this project, visual servoing acts as the "Fine Control" layer, taking over after coarse global detection to ensure high-precision alignment.

The objective is to minimize an error vector $e(t) = s - s^*$, representing the difference between current visual features (s) and desired features (s^*). The system follows a structured flow: perception of local corners via the eye-in-hand camera, target feature generation, error calculation, and the transformation of image-plane errors into Cartesian velocity commands.

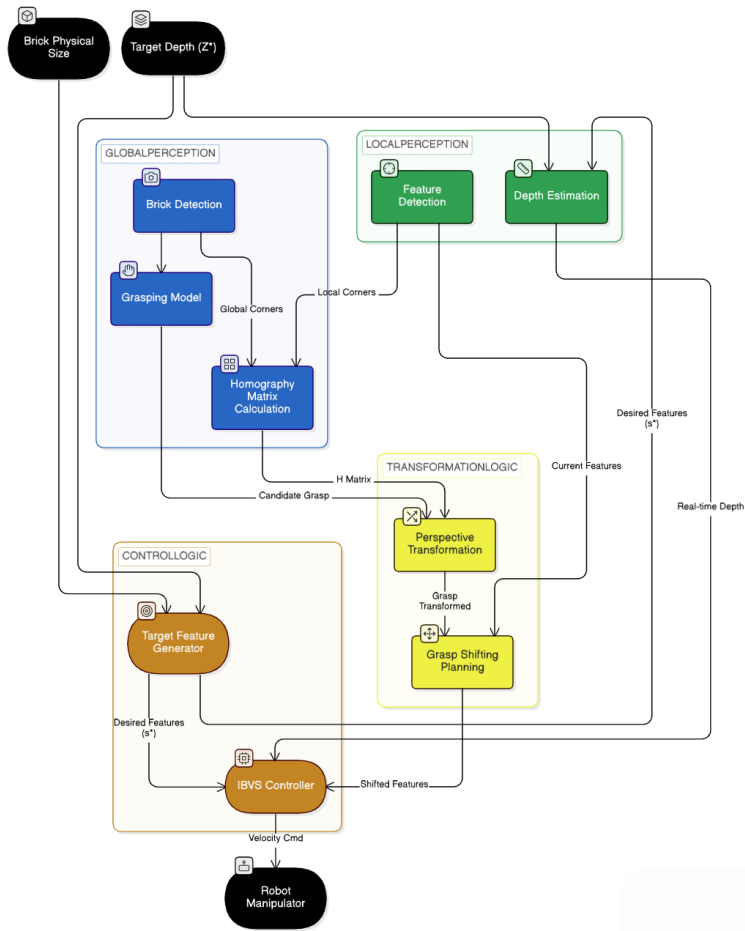


Figure 2.13: System Architecture Flowchart: Integration of Global Perception and Local IBVS Control.

2.8 Implementation Details

The implementation of the fine-control layer is structured into modular functional blocks that facilitate the transition from global environmental perception to high-precision local manipulation.

2.8.1 Local Perception and Depth Estimation

The local perception block provides high-frequency feedback for the closed-loop controller.

- Feature Detection:** A Region of Interest (ROI) tracker is employed to minimize computational load. It utilizes HSV color-space thresholding to isolate the brick and applies a fisheye distortion model to undistort coordinates. The tracker is initialized by the “seed” point provided by the transformation logic to ensure it locks onto the correct target.

- **Depth Estimation:** Real-time depth (Z_{curr}) is estimated by comparing the pixel area of the current features ($Area_{curr}$) to the dynamically calculated target area ($Area_{target}$). The system applies a smoothed area projection law :

$$Z_{curr} = Z_{target} \cdot \sqrt{\frac{Area_{target}}{Area_{curr}}} \quad (2.12)$$

2.8.2 Global Perception and Homography Matrix Calculation

The system initiates coordination by bridging the heterogeneous perspectives of the Realsense environment and ESP32 arm cameras.

- **Brick Detection and Grasping Model:** The environment camera identifies candidate targets and computes a candidate grasp point (u, v, z) where u , v represents the grasp points pixel coordinates and z represents the orientation angle .
- **Geometric Feature Matching:** To ensure cross-camera consistency, the system extracts rotated bounding box corners and calculates the length-to-width ratio of the detected brick. A candidate is only validated if its aspect ratio matches the local view within a 20% tolerance .
- **Homography Estimation:** Once a match is confirmed, a Homography matrix H is computed using the RANSAC algorithm. This matrix serves as the mathematical bridge to relate current image coordinates m_i to reference coordinates m_i^* via $m_i = Hm_i^*$.

2.8.3 Transformation Logic

This block translates global strategic goals into local tactical coordinates for the end-effector.

- **Perspective Transformation:** The candidate grasp point is projected into the local camera's image plane using the calculated matrix H . While the U and V pixel coordinates are transformed to account for the perspective shift, the orientation angle (Z) is passed directly to stabilize the goal state .
- **Grasp Shifting Planning:** The system “latches” onto the initial transformed point to set a local reference. It calculates a shift vector between the current brick centroid and the desired grasp pixel, effectively shifting the target features so the controller aligns the gripper with the specific grasp point rather than the brick center .

2.8.4 Control Logic and Feature Generation

The final stage generates the desired goal state and executes the motion command.

- **Target Feature Generator:** The system defines the desired state s^* by projecting a virtual 3D model of the brick into the image plane. It implements “Snap-to-Grid” logic to round the grasp angle to the nearest 90° , preventing rotational oscillations during the approach .

- **IBVS Controller:** The Image-Based Visual Servoing (IBVS) controller minimizes the error $e(t) = s - s^*$. It utilizes the Image Jacobian (L) to translate pixel errors into 6-DOF velocity commands, enabling the robot manipulator to achieve sub-millimeter alignment for the final pick operation .

2.9 Results and Evaluation

Experimental trials focused on stability and convergence. The pixel error norm successfully converged to < 5 pixels. The inclusion of RANSAC ensured sensor noise did not break the tracking lock, and fisheye calibration significantly reduced systematic bias.

Bibliography

- [1] S. Hutchinson, G. D. Hager, and P. I. Corke, "A Tutorial on Visual Servo Control," *IEEE Transactions on Robotics and Automation*, vol. 12, no. 5, pp. 651-670, 1996.
- [2] Y. Fang, D. M. Dawson, W. E. Dixon, and M. S. de Queiroz, "Homography-Based Visual Servoing of Wheeled Mobile Robots," *Proc. 41st IEEE Conf. Decision and Control*, 2002.
- [3] P. M. Khiabani, et al., "Visual Servoing Simulator by Using ROS and Gazebo," *4th Int. Conf. on Robotics and Mechatronics (ICRoM)*, 2016.
- [4] Y. Bai, et al., "EA-CTFVS: An Environment-Agnostic Coarse-to-Fine Visual Servoing Method for Sub-Millimeter-Accurate Assembly," *Actuators*, vol. 12, no. 1, 2023.
- [5] A. G. Schiano, et al., "A Review of Homography Estimation: Advances and Challenges," *MDPI: Sensors*, vol. 23, 2023.